

OK — je te fais une **fiche de spec complète**, notebook par notebook, avec pour **chaque fonction** :

- **but / logique**
- **inputs / outputs**
- **dépendances** : quelles fonctions déjà codées (00/01) elle appelle

Je pars de ta structure restante :

- 02_trading_environment.ipynb
- 03_quantile_regression.ipynb
- 04_actor_critic_networks.ipynb
- 05_d4pg_qr_algorithm.ipynb
- main.ipynb

(00 et 01 = déjà faits)

02_trading_environment.ipynb

Logique du notebook

Assembler vos briques 00+01 en un vrai **MDP exécutable** : reset() + step(action) qui simule **1 jour** de trading (arrivées → hedge → delta-neutral (implicite) → évolution marché → reward → next state).

Fonctions à coder

1) init_env(params, seed=None)

But : créer une structure d'environnement (dict) + RNG.

Inputs

- params: dict (r,q,v,rho, lambda_day, kappa, T_days, dt, T_client_days, T_hedge_days, S0, sigma0, etc.)
- seed: int|None

Outputs

- env: dict (avec au moins t, S, sigma, portfolio, P_prev, params)

Dépendances (00/01)

- aucune obligatoire (juste numpy RNG)
-

2) env_reset(env, S0=None, sigma0=None, seed=None)

But : reset épisode, portefeuille vide, calcule l'état initial.

Inputs

- env: dict
- S0: float|None (sinon env["params"]["S0"])
- sigma0: float|None
- seed: int|None

Outputs

- state: np.ndarray shape (5,)
- env: dict (mis à jour)

Dépendances

- (01) portfolio_greeks
 - (01) hedging_option
 - (01) build_state
-

3) simulate_client_flow_one_day(portfolio, S, params)

But : simuler arrivées clients du jour et append au portefeuille.

Inputs

- portfolio: list[dict]
- S: float
- params: dict (doit contenir lambda_day + pricing params)

Outputs

- portfolio: list[dict] (modifié)

Dépendances

- (01) sample_client_orders
- (01) create_client_option

4) mark_to_market_update(env)

But : faire avancer le **marché** d'un jour : S, σ via SABR discret.

Inputs

- env: dict

Outputs

- S_new: float
- sigma_new: float

Dépendances

- (00) correlated_normals
- (00) simulate_sabr_step
(ou tu peux appeler directement *simulate_sabr_step* en générant $z1, z2$ ici)

5) advance_portfolio_one_day(portfolio, dt)

But : décrémenter maturités d'un jour.

Inputs

- portfolio: list
- dt: float

Outputs

- portfolio: list

Dépendances

- (01) update_time_to_maturity

6) compute_H_max(port_greeks, hedge_greeks)

But : wrapper clair (si tu veux) pour “maximum hedge permitted”.

Inputs

- port_greeks: dict (gamma, vega)
- hedge_greeks: dict (gamma, vega)

Outputs

- H_max: float

Dépendances

- (01) max_hedge_position (*si tu gardes le même nom, tu n'as pas besoin de ce wrapper*)
-

7) env_step(env, action)

But : 1 journée complète (cœur du MDP).

Inputs

- env: dict
- action: float (dans [0,1])

Outputs

- next_state: np.ndarray (5,)
- reward: float
- done: bool
- info: dict (debug: H, H_max, cost, P_prev, P_curr, gamma_before/after, vega_before/after)

Dépendances

- (01) sample_client_orders, create_client_option (via simulate_client_flow_one_day)
- (01) portfolio_value
- (01) portfolio_greeks
- (01) hedging_option
- (01) max_hedge_position
- (01) apply_hedge
- (01) transaction_cost / compute_reward
- (01) build_state
- (01) update_time_to_maturity
- (00) simulate_sabr_step + correlated_normals (via mark_to_market_update)

Logique (fidèle papier)

1. Arrivées clients (Poisson) → update portefeuille
 2. Calcul grecs portefeuille (avant hedge)
 3. Définir option hedge ATM (greecs+prix)
 4. Calcul $H_{\max}$, puis $H = \text{action} * H_{\max}$
 5. Market move (S, σ) d'un jour
 6. Update maturités, expirations
 7. $P_{t-1} \rightarrow P_t$ via portfolio_value
 8. Reward $R_t = (P_t - P_{t-1}) - \kappa |V_t H_t|$
 9. Next state construit avec greecs recalculés
-

03_quantile_regression.ipynb

Logique du notebook

Tout ce qui concerne **QR** + **objectifs** (VaR/CVaR/mean-std) à partir d'un vecteur de quantiles. Indépendant du marché.

Fonctions à coder

1) make_quantile_grid(M)

Inputs M: int

Outputs taus: torch.Tensor (M,) ou np

Dépendances

- aucune (torch/np seulement)
-

2) huber_loss(u, k)

Inputs

- u: torch.Tensor
- k: float

Outputs

- L: torch.Tensor (même shape)

Dépendances

- aucune
-

3) `quantile_huber_loss(y, z, taus, k)`

But : loss critic QR (celle du papier).

Inputs

- `y`: torch.Tensor (B, M) (cibles quantiles)
- `z`: torch.Tensor (B, M) (prédits quantiles)
- `taus`: torch.Tensor (M,)
- `k`: float

Outputs

- `loss`: torch.Tensor scalaire

Dépendances

- (03) `huber_loss`
-

4) `var_from_quantiles(quantiles, alpha=0.95)`

Inputs

- `quantiles`: torch.Tensor (B,M) ou (M,)
- `alpha`: float

Outputs

- `var`: torch.Tensor (B,) ou float

Dépendances

- aucune
-

5) `cvar_from_quantiles(quantiles, alpha=0.95)`

Inputs idem

Outputs

- `cvar`: torch.Tensor (B,) ou float

Dépendances

- (03) var_from_quantiles (optionnel)
-

6) mean_std_objective_from_quantiles(quantiles, lambda_std=1.645)

Inputs

- quantiles: torch.Tensor (B,M)
- lambda_std: float

Outputs

- obj: torch.Tensor (B,)

Dépendances

- aucune
-

7) objective_from_quantiles(quantiles, objective, alpha=0.95, lambda_std=1.645)

Inputs

- quantiles: torch.Tensor (B,M)
- objective: str in {"mean_std","var","cvar"}
- alpha, lambda_std

Outputs

- obj: torch.Tensor (B,)

Dépendances

- (03) var_from_quantiles, cvar_from_quantiles, mean_std_objective_from_quantiles
-

04_actor_critic_networks.ipynb

Logique du notebook

Définir les réseaux PyTorch :

- Actor : $s \mapsto a \in [0,1]$
- Critic : $(s, a) \mapsto \text{quantiles } (B, M)$

Fonctions à coder

1) build_actor(state_dim=5, hidden=256, n_layers=3)

Outputs actor: nn.Module

Sortie forward : action: (B,1) borné [0,1]

Dépendances

- aucune (torch only)

2) build_critic_quantile(state_dim=5, action_dim=1, M=100, h1=512, h2=512, h3=256)

Outputs critic: nn.Module

Sortie forward : quantiles: (B,M)

Dépendances

- aucune

3) actor_forward(actor, state)

Inputs state: torch.Tensor (B,5)

Outputs action: torch.Tensor (B,1)

Dépendances

- (04) build_actor (le module)

4) critic_forward(critic, state, action)

Outputs quantiles: torch.Tensor (B,M)

Dépendances

- (04) build_critic_quantile

05_d4pg_qr_algorithm.ipynb

Logique du notebook

Implémenter l'entraînement :

- rollout → replay buffer
 - update critic QR (quantile huber)
 - update actor via objectif $f(Z)$
 - soft update targets
-

Fonctions / classes à coder

1) ReplayBuffer(capacity, state_dim, action_dim, M=None)

Méthodes

- add(s,a,r,s_next,done)
- sample(batch_size) → tensors
- __len__

Dépendances

- aucune (numpy/torch)
-

2) soft_update(target, online, tau)

Inputs

- target: nn.Module
- online: nn.Module
- tau: float

Outputs None (in-place)

Dépendances

- aucune
-

3) compute_target_quantiles(critic_target, actor_target, s_next, r, done, gamma)

But : construire $Y = r + \gamma Z(s', \pi(s'))$ (quantiles).

Inputs

- critic_target: nn.Module
- actor_target: nn.Module

- s_next: torch.Tensor (B,5)
- r: torch.Tensor (B,1)
- done: torch.Tensor (B,1) (0/1)
- gamma: float

Outputs

- Y: torch.Tensor (B,M)

Dépendances

- (04) actor_forward
- (04) critic_forward

4) update_critic(critic, optim_c, s, a, Y, taus, k)

Inputs

- critic, optim_c
- s: (B,5), a:(B,1)
- Y:(B,M) target
- taus:(M,)
- k: float

Outputs

- loss_value: float

Dépendances

- (04) critic_forward
- (03) quantile_huber_loss

5) update_actor(actor, optim_a, critic, s, objective, alpha=0.95, lambda_std=1.645)

But : minimiser $f(Z(s, \pi(s)))$.

Inputs

- actor, optim_a, critic
- s: (B,5)

- objective: str
- alpha, lambda_std

Outputs

- loss_value: float

Dépendances

- (04) actor_forward
- (04) critic_forward
- (03) objective_from_quantiles

6) rollout_one_episode(env, actor, noise_std=0.1)

But : générer une trajectoire 30 jours.

Inputs

- env (de 02)
- actor (04)
- noise_std: float (exploration)

Outputs

- episode_return: float
- traj: dict (list states/actions/rewards) *ou* transitions directement

Dépendances

- (02) env_reset, env_step
- (04) actor_forward (ou direct actor(state))

7) train(env, actor, critic, actor_t, critic_t, buffer, n_episodes, batch_size, updates_per_step, gamma, tau_soft, objective)

Outputs

- history: dict (returns, critic_loss, actor_loss, metrics)

Dépendances

- (05) rollout_one_episode

- (05) compute_target_quantiles
 - (05) update_critic
 - (05) update_actor
 - (05) soft_update
-

main.ipynb

Logique du notebook

Orchestration :

- set params
 - lancer entraînements pour objectives (mean-std / VaR / CVaR)
 - évaluer politiques
 - plots : distributions de pertes, profils de hedge, coûts vs risque
-

Fonctions (optionnelles mais propres)

1) run_experiment(params, objective, seed, n_episodes)

Outputs

- trained: dict (actor, critic, history)

Dépendances

- (02) init/reset/step via env
 - (04) build_actor/build_critic
 - (05) ReplayBuffer/train
-

2) evaluate_policy(env, actor, n_eval=200)

Outputs

- metrics: dict (mean, std, VaR, CVaR, mean_cost)
- samples: np.ndarray (loss samples)

Dépendances

- (02) env_reset, env_step

- (04) actor forward
 - (03) `var_from_quantiles/cvar_from_quantiles` *si tu estimes via quantiles* (sinon tu estimes empiriquement sur samples)
-

3) `plot_training_curves(history)`

Outputs plots

Dépendances

- matplotlib only
-

4) `plot_loss_distribution(samples_dict)`

Outputs plots (hist/cdf)

Dépendances

- matplotlib/numpy
-

Remarque importante sur les dépendances “import notebook”

Vu ton souci `import_ipynb` (exécute tout), le plus propre est :

- mettre toutes les fonctions “core” de 00/01/02/... dans des `.py` (ou notebooks *sans cellules exécutables*)
 - garder les tests/plots uniquement dans `main.ipynb`
-

Si tu veux, prochaine étape : je te propose une **arborescence exacte** (noms de fichiers + où mettre quoi) + l'ordre de développement pour que vous puissiez coder à trois sans vous marcher dessus (Idriss / Ghali / toi).